

Package ‘LSFoFQR’

May 21, 2026

Type Package

Title Locally Sparse Function-on-Function Quantile Regression

Version 0.1.0

Description Functions for fitting locally sparse simultaneous function-on-function quantile regression models. The package implements a three-stage estimation framework for functional responses and functional predictors, where the quantile-varying regression kernel is represented by tensor-product B-spline bases over the predictor, response, and quantile domains. Smoothness is controlled through directional P-spline penalties, while local sparsity is induced by an adaptive multiplicity-balanced overlapping group-Lasso penalty acting on cuboidal blocks of spline coefficients. Estimation proceeds through a smooth pilot fit, an adaptive locally sparse fit, and a constrained refit on the selected active coefficient set. The package provides functions for constructing the required B-spline bases, fitting the proposed model, selecting tuning parameters by a QBIC-type criterion, extracting estimated coefficient surfaces, predicting conditional quantile response curves for new functional predictors, and generating simulated data from a functional location-scale model with known quantile-varying coefficient surfaces.

Depends R (>= 3.5.0)

Imports Matrix,
MASS,
splines,
stats

License GPL-3

Encoding UTF-8

LazyData true

ByteCompile true

RoxygenNote 7.3.2

Contents

coef.sfqfofr	2
default_tuning	5
generate_sfqfofr_data	7
make_sfqfofr_basis	11

make_tuning_grid	15
predict.sfqfofr	18
sfqfofr	22

Index	29
--------------	-----------

coef.sfqfofr	<i>Extract Coefficient Surfaces from an sfqfofr Fit</i>
--------------	---

Description

Extracts the estimated coefficient surface from a fitted simultaneous function-on-function quantile regression model of class "sfqfofr". The returned object is the estimated regression kernel $\hat{\beta}(t, s, u)$ evaluated on the predictor grid, response grid, and quantile grid used in the fitted model.

Usage

```
## S3 method for class sfqfofr
coef(object, tau_index = NULL, ...)
```

Arguments

object	A fitted model object of class "sfqfofr", typically returned by sfqfofr . The object must contain the refitted coefficient vector, the B-spline basis objects, and the grids used for estimation.
tau_index	Optional integer index specifying the quantile level for which the coefficient surface should be returned. If NULL, the full three-dimensional array of coefficient surfaces over all quantile levels is returned. If supplied, tau_index should be an integer between 1 and length(object\$grids\$tau).
...	Additional arguments passed to the S3 method. Currently unused.

Details

The fitted simultaneous function-on-function quantile regression model has the form

$$Q_{Y(s)|X}(u | X) = \alpha(s, u) + \int_{\mathcal{T}} X(t) \beta(t, s, u) dt, \quad (s, u) \in \mathcal{S} \times \mathcal{U},$$

where $X(t)$ is the functional predictor, $Y(s)$ is the functional response, $\alpha(s, u)$ is the quantile-specific intercept surface, and $\beta(t, s, u)$ is the quantile-varying function-on-function regression kernel. The quantity $\beta(t, s, u)$ measures the contribution of the predictor trajectory at location t to the u th conditional quantile of the response at location s .

In the fitted model, the coefficient kernel is approximated by a tensor-product B-spline expansion,

$$\beta(t, s, u) \approx \sum_{a=1}^{K_t} \sum_{b=1}^{K_s} \sum_{c=1}^{K_u} \theta_{abc} b_a(t) c_b(s) d_c(u),$$

where $\{b_a(t)\}$, $\{c_b(s)\}$, and $\{d_c(u)\}$ are B-spline bases over the predictor, response, and quantile domains, respectively. The coefficient tensor θ_{abc} is estimated by the three-stage procedure implemented in [sfqfofr](#): a smooth pilot fit, an adaptive locally sparse fit, and a constrained refit on the selected active coefficient set.

This function reconstructs the estimated coefficient surface $\hat{\beta}(t, s, u)$ from the final refitted coefficient vector. If `tau_index = NULL`, the function evaluates the fitted kernel at all quantile levels in `object$grids$tau`. If `tau_index` is supplied, only the corresponding quantile-specific coefficient surface is returned.

The returned coefficient surface is evaluated on the same predictor and response grids used in the fitted object. It is therefore directly suitable for plotting, visual inspection of locally inactive regions, and interpretation of the lag-response-quantile dependence structure. In particular, regions where the estimated surface is exactly zero correspond to regions selected as locally inactive by the overlapping group-Lasso component of the estimator.

Value

If `tau_index = NULL`, a three-dimensional numeric array with dimensions

$$\text{length}(\text{object\$grids\$t}) \times \text{length}(\text{object\$grids\$s}) \times \text{length}(\text{object\$grids\$tau}).$$

The first dimension corresponds to the predictor-domain grid, the second to the response-domain grid, and the third to the quantile grid.

If `tau_index` is supplied, the function returns a three-dimensional array with the same first two dimensions and a singleton third dimension. Thus, the returned object has dimension

$$\text{length}(\text{object\$grids\$t}) \times \text{length}(\text{object\$grids\$s}) \times 1.$$

Note

This function extracts only the estimated coefficient kernel $\hat{\beta}(t, s, u)$. To obtain predicted conditional quantile response curves for new functional predictors, use [predict.sfqfofr](#).

Author(s)

Ufuk Beyaztas

Examples

```
## Grids and basis

t_grid <- seq(0, 1, length.out = 101)
s_grid <- seq(0, 1, length.out = 101)
tau_grid <- c(0.10, 0.35, 0.50, 0.65, 0.80, 0.90)

bases <- make_sfqfofr_basis(
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  kt = 11,
  ks = 11,
  ku = length(tau_grid),
  kas = 5,
  kau = length(tau_grid),
  degree_t = 2,
  degree_s = 2,
  degree_u = 2
)
```

```

## Generate data

dat <- generate_sfqfofr_data(
  n_train = 100,
  n_test = 1000,
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  bases = bases,
  error_type = "gaussian",
  support_design = "sparse",
  integration = "trapezoid"
)

## Sparse-design-friendly tuning values

tuning_sparse_example <- list(
  lam_init_b = 2.0e-04,
  lam_init_a = 5.0e-05,

  lam_sp_b   = 7.5e-05,
  lam_sp_a   = 3.0e-05,
  lam_sp_g   = 2.5e-02,

  lam_rf_b   = 3.5e-05,
  lam_rf_a   = 2.5e-05,

  active_tol  = 0.32,
  eta_adapt   = 1.5,
  active_method = "fixed_or_gap",
  min_gap     = 0.02
)

## Fit sfqfofr model

fit <- sfqfofr(
  Xmat = dat$train$X,
  Ymat = dat$train$Y,
  grids = list(t = t_grid, s = s_grid, tau = tau_grid),
  bases = bases,
  tuning = FALSE,
  base_tuning = tuning_sparse_example,
  integration = "trapezoid"
)

## Extract true and estimated beta(t,s,u) at u = 0.50

tau_id <- which.min(abs(tau_grid - 0.50))

beta_true_med <- dat$truth$beta_arr[, , tau_id]
beta_hat_med  <- coef(fit, tau_index = tau_id)[, , 1]

## Common z-axis range for fair visual comparison

zlim_common <- range(c(beta_true_med, beta_hat_med), na.rm = TRUE)

zpad <- 0.05 * diff(zlim_common)

```

```

if (!is.finite(zpad) || zpad == 0) zpad <- 0.1

zlim_common <- zlim_common + c(-zpad, zpad)

## Plot true and estimated coefficient surfaces side by side

op <- par(
  mfrow = c(1, 2),
  mar = c(2.5, 2.5, 3.0, 1.0),
  oma = c(0.2, 0.2, 0.2, 0.2)
)

persp(
  x = t_grid,
  y = s_grid,
  z = beta_true_med,
  theta = 35,
  phi = 25,
  expand = 0.75,
  col = "gray92",
  border = "gray70",
  ticktype = "detailed",
  xlab = "t",
  ylab = "s",
  zlab = expression(beta(t, s)),
  zlim = zlim_common,
  main = expression(True~~beta(t, s)~~at~~u == 0.50)
)

persp(
  x = t_grid,
  y = s_grid,
  z = beta_hat_med,
  theta = 35,
  phi = 25,
  expand = 0.75,
  col = "gray92",
  border = "gray70",
  ticktype = "detailed",
  xlab = "t",
  ylab = "s",
  zlab = expression(hat(beta)(t, s)),
  zlim = zlim_common,
  main = expression(Estimated~~beta(t, s)~~at~~u == 0.50)
)

par(op)

```

default_tuning

Default Tuning Parameters for LSFoFQR

Description

Constructs a default tuning-parameter vector for the proposed locally sparse simultaneous function-on-function quantile regression estimator. The returned list contains the smoothing, sparsity, refit-

ting, adaptivity, and support-screening parameters used by [sfqfofr](#).

Usage

```
default_tuning(n_train)
```

Arguments

n_train Positive integer giving the number of independent training curves. The value is used to mildly scale the group-sparsity parameter and to determine the default adaptive-weight exponent and active-block threshold.

Details

The proposed estimator is computed through a three-stage procedure. First, a smooth pilot fit is obtained using only smoothness and ridge regularization. Second, an adaptive locally sparse estimator is computed by adding a multiplicity-balanced overlapping group-Lasso penalty to the coefficient tensor. Third, a constrained refit is performed on the active coefficient set selected from the sparse fit.

The tuning vector returned by `default_tuning()` contains the numerical parameters controlling these three stages. The pilot-stage smoothing parameters are `lam_init_b` for the coefficient kernel and `lam_init_a` for the intercept surface. The sparse-stage smoothing parameters are `lam_sp_b` and `lam_sp_a`, while `lam_sp_g` controls the adaptive overlapping group-Lasso penalty. The refit-stage smoothing parameters are `lam_rf_b` and `lam_rf_a`. The support-screening step is controlled by `active_tol`, `active_method`, and `min_gap`. The adaptive-weight exponent is given by `eta_adapt`.

The simultaneous function-on-function quantile regression model is

$$Q_{Y(s)|X}(u | X) = \alpha(s, u) + \int_{\mathcal{T}} X(t)\beta(t, s, u) dt,$$

where $\beta(t, s, u)$ is represented by a tensor-product B-spline expansion over the predictor, response, and quantile domains. The local-sparsity penalty acts on overlapping cuboidal blocks of the coefficient tensor. Therefore, the tuning parameters returned by this function determine the balance between smoothness, sparsity, and post-selection refitting in the estimated kernel $\hat{\beta}(t, s, u)$.

The default values are intended to provide stable initial values for practical use and for construction of tuning grids. They may not be optimal for every data set. Users can modify the returned list directly or use it as the center of a candidate grid through [make_tuning_grid](#).

Value

A named list with the following components:

<code>lam_init_b</code>	Pilot-stage smoothing parameter for the coefficient kernel $\beta(t, s, u)$.
<code>lam_init_a</code>	Pilot-stage smoothing parameter for the intercept surface $\alpha(s, u)$.
<code>lam_sp_b</code>	Sparse-stage smoothing parameter for the coefficient kernel.
<code>lam_sp_a</code>	Sparse-stage smoothing parameter for the intercept surface.
<code>lam_sp_g</code>	Sparse-stage adaptive overlapping group-Lasso parameter. Larger values produce stronger local sparsity in the estimated coefficient kernel. The default value is scaled by $\sqrt{n_{\text{train}}/100}$.
<code>lam_rf_b</code>	Refit-stage smoothing parameter for the coefficient kernel after support selection.

lam_rf_a	Refit-stage smoothing parameter for the intercept surface after support selection.
active_tol	Threshold used in the hybrid active-block screening rule. Blocks with sufficiently large hybrid support scores are retained for the constrained refit. The default is bounded between 0.03 and 0.45.
eta_adapt	Adaptive-weight exponent used in the group penalty. Larger values place more emphasis on the pilot fit when constructing adaptive group weights.
active_method	Character string specifying the active-block selection rule. The default "fixed_or_gap" combines a fixed threshold with a gap-based fallback.
min_gap	Minimum gap required for the gap-based active-block screening rule.

Note

The values returned by this function are default starting values, not universal optimal tuning parameters. For formal analysis, the tuning parameters should be selected by the QBIC-type criterion implemented in the fitting procedure or by a user-defined tuning grid.

Author(s)

Ufuk Beyaztas

Examples

```
## Default tuning for a sample of 100 functional observations
tun100 <- default_tuning(n_train = 100)
tun100

## Default tuning for a larger sample
tun250 <- default_tuning(n_train = 250)
tun250

## Modify one component manually
tun100$lam_sp_g <- 2 * tun100$lam_sp_g

## Use the default vector as the center of a tuning grid
grid <- make_tuning_grid(n_train = 100, base_tuning = tun100)
length(grid)
```

generate_sfqfofr_data *Generate Simulated SFQFoFR Data*

Description

Generates simulated functional predictor and functional response data for the simultaneous function-on-function quantile regression model. The data-generating mechanism follows a location-scale functional model with a known quantile-varying coefficient surface, allowing the fitted coefficient kernel and conditional quantile surfaces to be evaluated against the truth.

Usage

```
generate_sfqfofr_data(
  n_train = 100,
  n_test = 1000,
  t_grid,
  s_grid,
  tau_grid,
  bases,
  error_type = c("gaussian", "t3", "cauchy"),
  support_design = c("nonsparse", "nonsparse_noncrossing", "sparse"),
  integration = c("trapezoid", "simpson", "auto")
)
```

Arguments

n_train	Positive integer giving the number of training functional observations.
n_test	Positive integer giving the number of test functional observations.
t_grid	Numeric vector giving the grid over the predictor domain $\mathcal{T}$. The grid must be strictly increasing.
s_grid	Numeric vector giving the grid over the response domain $\mathcal{S}$. The grid must be strictly increasing.
tau_grid	Numeric vector of quantile levels. All elements should lie strictly between 0 and 1.
bases	A list of B-spline basis matrices, typically returned by make_sfqfofr_basis . The list must contain the components Bt, Bs, Bu, Bsa, and Bua.
error_type	Character string specifying the marginal innovation distribution. Available options are "gaussian", "t3", and "cauchy". The default is "gaussian".
support_design	Character string specifying the coefficient-surface design. Available options are "nonsparse", "nonsparse_noncrossing", and "sparse".
integration	Numerical integration rule used to approximate functional inner products. Available options are "trapezoid", "simpson", and "auto". The default is "trapezoid", which is recommended for general grids because it works for arbitrary grid lengths.

Details

The function generates data from a functional location-scale model whose conditional quantile function has the simultaneous function-on-function quantile regression form

$$Q_{Y(s)|X}(u | X) = \alpha_0(s, u) + \int_{\mathcal{T}} X(t) \beta_0(t, s, u) dt.$$

The functional predictor is generated from a finite Karhunen–Loeve-type expansion,

$$X_i(t) = \sum_{k=1}^6 \xi_{ik} \phi_k(t) + \delta_i(t),$$

where the basis functions are trigonometric functions and the scores have decreasing standard deviations. The generated predictor trajectories are then centered pointwise.

The functional response is generated from the location-scale model

$$Y_i(s) = \mu_i(s) + \sigma_i(s)\varepsilon_i(s),$$

where

$$\mu_i(s) = a_\mu(s) + \int_{\mathcal{T}} X_i(t)\Theta_\mu(t, s) dt,$$

and

$$\sigma_i(s) = a_\sigma(s) + \int_{\mathcal{T}} X_i(t)\Theta_\sigma(t, s) dt.$$

The innovation process $\varepsilon_i(s)$ has exponential correlation over the response domain. The marginal innovation distribution is selected by `error_type`. The Gaussian setting provides a light-tailed case, the Student- t_3 setting provides a moderately heavy-tailed case, and the Cauchy setting provides an extreme heavy-tailed stress test.

Because the innovation distribution is symmetric, the conditional quantile surface is available in closed form:

$$Q_{Y_i(s)|X_i}(u | X_i) = a_\mu(s) + q_u a_\sigma(s) + \int_{\mathcal{T}} X_i(t) \{\Theta_\mu(t, s) + q_u \Theta_\sigma(t, s)\} dt,$$

where q_u is the u th quantile of the selected innovation distribution. Consequently,

$$\alpha_0(s, u) = a_\mu(s) + q_u a_\sigma(s), \quad \beta_0(t, s, u) = \Theta_\mu(t, s) + q_u \Theta_\sigma(t, s).$$

The returned object contains both the observed training and test samples and the corresponding true conditional quantile surfaces.

Three coefficient-support designs are available. The "nonsparse" design uses a dense smooth coefficient surface that changes sign. The "nonsparse_noncrossing" design uses a dense smooth coefficient surface that is constrained to remain nonzero over the considered quantile range. The "sparse" design generates a locally sparse coefficient surface with separated active regions and extended inactive regions. The sparse design requires at least 11 basis functions in both the predictor and response directions.

Value

A list with the following components:

<code>train</code>	A list containing the training data: • <code>X</code>: an <code>n_train</code> by <code>length(t_grid)</code> matrix of functional predictor observations; • <code>Y</code>: an <code>n_train</code> by <code>length(s_grid)</code> matrix of functional response observations; • <code>Qtrue</code>: an array of dimension <code>n_train</code> by <code>length(s_grid)</code> by <code>length(tau_grid)</code> containing the true conditional quantile response surfaces for the training observations.
<code>test</code>	A list containing the test data: • <code>X</code>: an <code>n_test</code> by <code>length(t_grid)</code> matrix of functional predictor observations; • <code>Y</code>: an <code>n_test</code> by <code>length(s_grid)</code> matrix of functional response observations; • <code>Qtrue</code>: an array of dimension <code>n_test</code> by <code>length(s_grid)</code> by <code>length(tau_grid)</code> containing the true conditional quantile response surfaces for the test observations.

truth A list containing the true data-generating objects:

- Theta_mean: coefficient matrix for the location component;
- Theta_scale: coefficient matrix for the scale component;
- theta_true_arr: tensor of true B-spline coefficients;
- alpha_coef: matrix of true intercept-surface coefficients;
- beta_arr: array containing the true coefficient surfaces $\beta_0(t, s, u)$ on the observed grids;
- alpha_arr: matrix containing the true intercept surface $\alpha_0(s, u)$;
- mean_curve: generated conditional location curves;
- scale_curve: generated conditional scale curves;
- q_tau: distributional quantiles corresponding to tau_grid;
- support_design: support design used in the simulation;
- error_type: innovation distribution used in the simulation.

Author(s)

Ufuk Beyaztas

Examples

```
t_grid <- seq(0, 1, length.out = 101)
s_grid <- seq(0, 1, length.out = 101)
tau_grid <- c(0.10, 0.35, 0.50, 0.65, 0.80, 0.90)
```

```
bases <- make_sfqfofr_basis(
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  kt = 11,
  ks = 11,
  ku = length(tau_grid),
  kas = 5,
  kau = length(tau_grid),
  degree_t = 2,
  degree_s = 2,
  degree_u = 2
)
```

```
dat <- generate_sfqfofr_data(
  n_train = 100,
  n_test = 1000,
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  bases = bases,
  error_type = "gaussian",
  support_design = "sparse",
  integration = "trapezoid"
)
```

```
dim(dat$train$X)
dim(dat$train$Y)
```

```
dim(dat$train$Qtrue)
dim(dat$truth$beta_arr)
```

make_sfqfofr_basis	<i>Construct B-Spline Bases for SFQFoFR</i>
--------------------	---

Description

Constructs the B-spline basis matrices used to fit the locally sparse simultaneous function-on-function quantile regression model. The function creates separate bases for the predictor domain, response domain, quantile domain, and the intercept surface.

Usage

```
make_sfqfofr_basis(
  t_grid,
  s_grid,
  tau_grid,
  kt = 11,
  ks = 11,
  ku = NULL,
  kas = 5,
  kau = NULL,
  degree_t = 2L,
  degree_s = 2,
  degree_u = 2
)
```

Arguments

t_grid	Numeric vector giving the grid over the predictor domain $\mathcal{T}$. The grid should be strictly increasing.
s_grid	Numeric vector giving the grid over the response domain $\mathcal{S}$. The grid should be strictly increasing.
tau_grid	Numeric vector of quantile levels over the quantile domain $\mathcal{U}$. All entries should lie strictly between 0 and 1.
kt	Number of B-spline basis functions used in the predictor direction t . The default is 11.
ks	Number of B-spline basis functions used in the response direction s . The default is 11.
ku	Number of B-spline basis functions used in the quantile direction u for the coefficient kernel. If NULL, it is set equal to <code>length(tau_grid)</code> .
kas	Number of B-spline basis functions used in the response direction for the intercept surface $\alpha(s, u)$. The default is 5.
kau	Number of B-spline basis functions used in the quantile direction for the intercept surface $\alpha(s, u)$. If NULL, it is set equal to <code>length(tau_grid)</code> .
degree_t	Polynomial degree of the B-spline basis in the predictor direction. The default is 2, corresponding to quadratic B-splines.

degree_s	Polynomial degree of the B-spline basis in the response direction. The default is 2, corresponding to quadratic B-splines.
degree_u	Polynomial degree of the B-spline basis in the quantile direction. The default is 2, corresponding to quadratic B-splines.

Details

The simultaneous function-on-function quantile regression model has the form

$$Q_{Y(s)|X}(u | X) = \alpha(s, u) + \int_{\mathcal{T}} X(t) \beta(t, s, u) dt, \quad (s, u) \in \mathcal{S} \times \mathcal{U}.$$

The coefficient kernel $\beta(t, s, u)$ is represented by a tensor-product B-spline expansion,

$$\beta(t, s, u) \approx \sum_{a=1}^{K_t} \sum_{b=1}^{K_s} \sum_{c=1}^{K_u} \theta_{abc} b_a(t) c_b(s) d_c(u),$$

where $b_a(t)$, $c_b(s)$, and $d_c(u)$ are B-spline bases over the predictor, response, and quantile domains, respectively.

The intercept surface is represented separately as

$$\alpha(s, u) \approx \sum_{b=1}^{L_s} \sum_{c=1}^{L_u} \xi_{bc} \tilde{c}_b(s) \tilde{d}_c(u).$$

This function constructs the basis matrices required for these two representations. The coefficient-kernel basis consists of Bt, Bs, and Bu. The intercept-surface basis consists of Bsa and Bua.

The tensor-product representation is used internally by `sfqfofr` to build the finite-dimensional linear predictor,

$$\eta_i(s, u) = \{\tilde{d}(u) \otimes \tilde{c}(s)\}^\top \xi + \{d(u) \otimes c(s) \otimes r_i\}^\top \theta,$$

where

$$r_i = \int_{\mathcal{T}} X_i(t) b(t) dt$$

is the predictor-basis score vector. The local support of the B-spline basis is also central to the local-sparsity mechanism: when a block of tensor-product basis coefficients is set to zero, the fitted coefficient kernel is exactly zero on the corresponding open tensor-product knot cells.

For the default quadratic B-splines, each basis function has compact local support over a small number of adjacent intervals. This property allows the overlapping group-Lasso penalty in `sfqfofr` to induce interpretable locally inactive regions in the estimated coefficient surface.

Value

A named list with the following components:

Bt	Matrix of B-spline basis evaluations over <code>t_grid</code> . It has dimension <code>length(t_grid)</code> by <code>kt</code> . This basis is used for the predictor direction of the coefficient kernel.
Bs	Matrix of B-spline basis evaluations over <code>s_grid</code> . It has dimension <code>length(s_grid)</code> by <code>ks</code> . This basis is used for the response direction of the coefficient kernel.
Bu	Matrix of B-spline basis evaluations over <code>tau_grid</code> . It has dimension <code>length(tau_grid)</code> by <code>ku</code> . This basis is used for the quantile direction of the coefficient kernel.
Bsa	Matrix of B-spline basis evaluations over <code>s_grid</code> . It has dimension <code>length(s_grid)</code> by <code>kas</code> . This basis is used for the response direction of the intercept surface.
Bua	Matrix of B-spline basis evaluations over <code>tau_grid</code> . It has dimension <code>length(tau_grid)</code> by <code>kau</code> . This basis is used for the quantile direction of the intercept surface.

Note

The default polynomial degree is 2, corresponding to quadratic B-splines. The locally sparse estimator uses overlapping cuboidal blocks of basis coefficients. For stable use of the default local-sparsity structure, the basis dimensions kt , ks , and ku should generally be at least 3.

When $ku = \text{NULL}$ or $kau = \text{NULL}$, the corresponding number of quantile-basis functions is set equal to $\text{length}(\text{tau_grid})$. This is convenient when the model is fitted on a modest fixed grid of quantile levels.

Author(s)

Ufuk Beyaztas

Examples

```
## Construct grids
t_grid  <- seq(0, 1, length.out = 101)
s_grid  <- seq(0, 1, length.out = 101)
tau_grid <- c(0.10, 0.35, 0.50, 0.65, 0.80, 0.90)
```

```
## Build B-spline bases
bases <- make_sfqfofr_basis(
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  kt = 11,
  ks = 11,
  ku = length(tau_grid),
  kas = 5,
  kau = length(tau_grid),
  degree_t = 2,
  degree_s = 2,
  degree_u = 2
)
```

```
## Inspect basis dimensions
dim(bases$Bt)
dim(bases$Bs)
dim(bases$Bu)
dim(bases$Bsa)
dim(bases$Bua)
```

```
## Grids and basis
```

```
t_grid  <- seq(0, 1, length.out = 101)
s_grid  <- seq(0, 1, length.out = 101)
tau_grid <- c(0.10, 0.35, 0.50, 0.65, 0.80, 0.90)
```

```
bases <- make_sfqfofr_basis(
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  kt = 11,
  ks = 11,
  ku = length(tau_grid),
  kas = 5,
```

```

    kau = length(tau_grid),
    degree_t = 2,
    degree_s = 2,
    degree_u = 2
  )

## Use the basis object in a small simulated example
dat <- generate_sfqfofr_data(
  n_train = 100,
  n_test = 1000,
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  bases = bases,
  error_type = "gaussian",
  support_design = "sparse",
  integration = "trapezoid"
)

tuning_sparse_example <- list(
  lam_init_b = 2.0e-04,
  lam_init_a = 5.0e-05,

  lam_sp_b = 7.5e-05,
  lam_sp_a = 3.0e-05,
  lam_sp_g = 2.5e-02,

  lam_rf_b = 3.5e-05,
  lam_rf_a = 2.5e-05,

  active_tol = 0.32,
  eta_adapt = 1.5,
  active_method = "fixed_or_gap",
  min_gap = 0.02
)

fit <- sfqfofr(
  Xmat = dat$train$X,
  Ymat = dat$train$Y,
  grids = list(t = t_grid, s = s_grid, tau = tau_grid),
  bases = bases,
  tuning = FALSE,
  base_tuning = tuning_sparse_example,
  integration = "trapezoid"
)

## Extract true and estimated beta(t,s,u) at u = 0.50
tau_id <- which.min(abs(tau_grid - 0.50))

beta_true_med <- dat$truth$beta_arr[, , tau_id]
beta_hat_med <- coef(fit, tau_index = tau_id)[, , 1]

## Common z-axis range for fair visual comparison
zlim_common <- range(c(beta_true_med, beta_hat_med), na.rm = TRUE)

```

```

zpad <- 0.05 * diff(zlim_common)
if (!is.finite(zpad) || zpad == 0) zpad <- 0.1

zlim_common <- zlim_common + c(-zpad, zpad)

## Plot true and estimated coefficient surfaces side by side

op <- par(
  mfrow = c(1, 2),
  mar = c(2.5, 2.5, 3.0, 1.0),
  oma = c(0.2, 0.2, 0.2, 0.2)
)

persp(
  x = t_grid,
  y = s_grid,
  z = beta_true_med,
  theta = 35,
  phi = 25,
  expand = 0.75,
  col = "gray92",
  border = "gray70",
  ticktype = "detailed",
  xlab = "t",
  ylab = "s",
  zlab = expression(beta(t, s)),
  zlim = zlim_common,
  main = expression(True~~beta(t, s)~~at~~u == 0.50)
)

persp(
  x = t_grid,
  y = s_grid,
  z = beta_hat_med,
  theta = 35,
  phi = 25,
  expand = 0.75,
  col = "gray92",
  border = "gray70",
  ticktype = "detailed",
  xlab = "t",
  ylab = "s",
  zlab = expression(hat(beta)(t, s)),
  zlim = zlim_common,
  main = expression(Estimated~~beta(t, s)~~at~~u == 0.50)
)

par(op)

```

Description

Constructs a finite candidate grid of tuning-parameter vectors for the proposed locally sparse simultaneous function-on-function quantile regression estimator. The grid is formed by multiplying a baseline tuning vector by user-specified smoothness and sparsity factors and by varying the active-block screening parameters.

Usage

```
make_tuning_grid(
  n_train,
  base_tuning = default_tuning(n_train),
  smooth_mult = c(0.5, 1.0, 2.0),
  group_mult = c(0.5, 1.0, 2.0),
  active_tol_grid = NULL,
  eta_grid = NULL,
  active_method = "fixed_or_gap",
  min_gap = 0.02
)
```

Arguments

n_train	Positive integer giving the number of independent training curves. This argument is used to construct base_tuning through default_tuning when base_tuning is not supplied.
base_tuning	Named list giving the baseline tuning vector around which the grid is constructed. By default, this is default_tuning(n_train).
smooth_mult	Numeric vector of multiplicative factors applied to all smoothness parameters. These factors are applied to lam_init_b, lam_init_a, lam_sp_b, lam_sp_a, lam_rf_b, and lam_rf_a.
group_mult	Numeric vector of multiplicative factors applied to the adaptive overlapping group-Lasso parameter lam_sp_g.
active_tol_grid	Optional numeric vector of active-block threshold values. If NULL, the grid is constructed automatically around base_tuning\$active_tol using the values base_tuning\$active_tol - 0.08, base_tuning\$active_tol, and base_tuning\$active_tol + 0.08, truncated to lie between 0.03 and 0.55.
eta_grid	Optional numeric vector of adaptive-weight exponents. If NULL, the value base_tuning\$eta_adapt is used.
active_method	Character string specifying the support-screening rule used in the constrained refit. The default is "fixed_or_gap".
min_gap	Positive numeric value giving the minimum gap required by the gap-based active-block screening rule. The default is 0.02.

Details

The proposed estimator is computed through three stages: a smooth pilot fit, an adaptive locally sparse fit, and a constrained refit on the selected active coefficient set. The tuning parameters control smoothness, local sparsity, adaptive weighting, and support screening across these stages.

Let

$$\zeta = (\lambda_{\alpha}^{(0)}, \lambda_{\beta}^{(0)}, \lambda_{\alpha}, \lambda_{\beta}, \tau_{\beta}, \lambda_{\alpha}^*, \lambda_{\beta}^*, \kappa, \eta)$$

denote a generic tuning vector. Here $\lambda_{\alpha}^{(0)}$ and $\lambda_{\beta}^{(0)}$ control the smooth pilot fit, λ_{α} , λ_{β} , and τ_{β} control the adaptive sparse fit, λ_{α}^* and λ_{β}^* control the constrained refit, κ is the active-block threshold, and η is the adaptive-weight exponent.

This function constructs a finite candidate set $\mathcal{Z}$ by scaling a baseline tuning vector. The smoothness multiplier `smooth_mult` is applied simultaneously to the pilot-stage, sparse-stage, and refit-stage smoothing parameters. The group multiplier `group_mult` is applied only to the adaptive overlapping group-Lasso parameter `lam_sp_g`. The threshold and adaptive exponent are varied through `active_tol_grid` and `eta_grid`.

The resulting list is designed for use in `sfqfofr` when `tuning = TRUE`. For each candidate tuning vector, the full three-stage estimation procedure is carried out, and the selected tuning vector is the one minimizing the QBIC-type criterion implemented internally by the fitting procedure.

Value

A list of tuning-parameter vectors. Each element of the returned list is itself a named list with the following components:

<code>lam_init_b</code>	Pilot-stage smoothing parameter for the coefficient kernel $\beta(t, s, u)$.
<code>lam_init_a</code>	Pilot-stage smoothing parameter for the intercept surface $\alpha(s, u)$.
<code>lam_sp_b</code>	Sparse-stage smoothing parameter for the coefficient kernel.
<code>lam_sp_a</code>	Sparse-stage smoothing parameter for the intercept surface.
<code>lam_sp_g</code>	Sparse-stage adaptive overlapping group-Lasso parameter controlling the strength of local sparsity.
<code>lam_rf_b</code>	Refit-stage smoothing parameter for the coefficient kernel.
<code>lam_rf_a</code>	Refit-stage smoothing parameter for the intercept surface.
<code>active_tol</code>	Threshold used in the hybrid active-block screening score.
<code>eta_adapt</code>	Adaptive-weight exponent used to construct the group-specific adaptive weights from the pilot fit.
<code>active_method</code>	Character string specifying the active-block screening rule.
<code>min_gap</code>	Minimum gap used by the gap-based part of the active-block screening rule.

Note

If `base_tuning` is supplied directly, `n_train` is not used inside the function body. However, when `base_tuning` is omitted, `n_train` must be provided so that `default_tuning` can construct the baseline tuning vector.

Author(s)

Ufuk Beyaztas

Examples

```
## Construct a default tuning grid for n = 100
grid1 <- make_tuning_grid(n_train = 100)
length(grid1)

## Inspect the first candidate tuning vector
grid1[[1]]
```

```
## Use a smaller grid for faster tuning
grid2 <- make_tuning_grid(
  n_train = 100,
  smooth_mult = c(1),
  group_mult = c(0.5, 1, 2),
  active_tol_grid = c(0.15, 0.25),
  eta_grid = c(1.25)
)
length(grid2)

## Start from a user-modified baseline tuning vector
base <- default_tuning(n_train = 100)
base$lam_sp_g <- 2 * base$lam_sp_g

grid3 <- make_tuning_grid(
  n_train = 100,
  base_tuning = base,
  smooth_mult = c(0.5, 1, 2),
  group_mult = c(1)
)
length(grid3)
```

predict.sfqfofr

Predict Conditional Quantile Curves from an sfqfofr Fit

Description

Computes predicted conditional quantile response curves for new functional predictors using a fitted locally sparse simultaneous function-on-function quantile regression model.

Usage

```
## S3 method for class sfqfofr
predict(object, Xnew, ...)
```

Arguments

object	A fitted model object of class "sfqfofr", typically returned by sfqfofr .
Xnew	Numeric matrix containing new functional predictor observations. Each row represents one new functional predictor curve, and each column corresponds to one point on the predictor grid object\$grids\$t. Thus, ncol(Xnew) must equal length(object\$grids\$t).
...	Additional arguments passed to the S3 method. Currently unused.

Details

The fitted model has the simultaneous function-on-function quantile regression form

$$Q_{Y(s)|X}(u | X) = \alpha(s, u) + \int_{\mathcal{T}} X(t) \beta(t, s, u) dt, \quad (s, u) \in \mathcal{S} \times \mathcal{U}.$$

For a new functional predictor $X^*(t)$, this function computes

$$\hat{Q}_{Y^*(s)|X^*}(u | X^*) = \hat{\alpha}(s, u) + \int_{\mathcal{T}} X^*(t) \hat{\beta}(t, s, u) dt$$

on the response grid and quantile grid stored in the fitted object.

Internally, the functional inner product is approximated by first computing the predictor-basis score vector

$$\hat{r}^* = \sum_j \delta_j X^*(t_j) b(t_j),$$

where δ_j are numerical integration weights and $b(t)$ is the B-spline basis over the predictor domain. The integration rule is inherited from the fitted object through `object$integration`. The default fitting rule is trapezoidal integration, which is suitable for general grids.

The coefficient kernel used in prediction is the final constrained-refit estimator stored in `object$fit_r`. Therefore, predicted quantile curves are based on the locally sparse, bias-reduced final estimator rather than on the pilot or sparse intermediate fits.

The prediction is simultaneous over all quantile levels in `object$grids$tau`. The output is an array whose third dimension indexes the quantile levels.

Value

A three-dimensional numeric array with dimensions

$$n_{\text{new}} \times \text{length}(\text{object\$grids\$s}) \times \text{length}(\text{object\$grids\$tau}).$$

The first dimension indexes new observations, the second dimension indexes the response-domain grid, and the third dimension indexes the quantile levels.

For example, `out[i, , r]` is the predicted conditional quantile response curve for the i th new predictor at the quantile level `object$grids$tau[r]`.

Note

The new predictor matrix X_{new} must be observed on the same predictor grid used to fit the model. If new curves are observed on a different grid, they should first be interpolated or registered to `object$grids$t`.

This function returns predicted conditional quantile curves. To extract the estimated coefficient kernel $\hat{\beta}(t, s, u)$, use `coef.sfqfofr`.

Author(s)

Ufuk Beyaztas

Examples

```
## Construct grids
t_grid  <- seq(0, 1, length.out = 101)
s_grid  <- seq(0, 1, length.out = 101)
tau_grid <- c(0.10, 0.35, 0.50, 0.65, 0.80, 0.90)

## Build basis matrices
bases <- make_sfqfofr_basis(
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  kt = 11,
  ks = 11,
  ku = length(tau_grid),
```

```

kas = 5,
kau = length(tau_grid),
degree_t = 2,
degree_s = 2,
degree_u = 2
)

## Generate a small data set
dat <- generate_sfqfofr_data(
  n_train = 100,
  n_test = 1000,
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  bases = bases,
  error_type = "gaussian",
  support_design = "sparse",
  integration = "trapezoid"
)

## Sparse-design-friendly tuning values
tuning_sparse_example <- list(
  lam_init_b = 2.0e-04,
  lam_init_a = 5.0e-05,

  lam_sp_b = 7.5e-05,
  lam_sp_a = 3.0e-05,
  lam_sp_g = 2.5e-02,

  lam_rf_b = 3.5e-05,
  lam_rf_a = 2.5e-05,

  active_tol = 0.32,
  eta_adapt = 1.5,
  active_method = "fixed_or_gap",
  min_gap = 0.02
)

## Fit sfqfofr model
fit <- sfqfofr(
  Xmat = dat$train$X,
  Ymat = dat$train$Y,
  grids = list(t = t_grid, s = s_grid, tau = tau_grid),
  bases = bases,
  tuning = FALSE,
  base_tuning = tuning_sparse_example,
  integration = "trapezoid"
)

## Predict conditional quantile response curves for test predictors
pred <- predict(fit, Xnew = dat$test$X)

## Dimensions: n_test by length(s_grid) by length(tau_grid)
dim(pred)

```

```

## Median quantile index
tau_id <- which.min(abs(tau_grid - 0.50))

## Select the first 10 test curves
id_plot <- 1:10

## True and estimated conditional median curves
Q_true_med <- dat$test$Qtrue[id_plot, , tau_id]
Q_hat_med <- pred[id_plot, , tau_id]

## Plot true and estimated median curves
op <- par(
  mfrow = c(2, 5),
  mar = c(3.0, 3.0, 2.2, 0.8),
  oma = c(0.2, 0.2, 1.0, 0.2)
)

for (ii in seq_along(id_plot)) {

  yy <- range(c(Q_true_med[ii, ], Q_hat_med[ii, ]), na.rm = TRUE)

  plot(
    s_grid,
    Q_true_med[ii, ],
    type = "l",
    lwd = 2,
    ylim = yy,
    xlab = "s",
    ylab = "Median curve",
    main = paste0("Test curve ", id_plot[ii])
  )

  lines(
    s_grid,
    Q_hat_med[ii, ],
    lwd = 2,
    lty = 2
  )

  legend(
    "topright",
    legend = c("True", "Estimated"),
    lty = c(1, 2),
    lwd = c(2, 2),
    bty = "n",
    cex = 0.75
  )
}

mtext(
  expression("True and estimated conditional median curves, " * u == 0.50),
  side = 3,
  outer = TRUE,
  line = -0.2,
  font = 2
)

```

par(op)

sfqfofr

Fit Locally Sparse Simultaneous Function-on-Function Quantile Regression

Description

Fits the proposed locally sparse simultaneous function-on-function quantile regression model. The method estimates a quantile-varying regression kernel $\beta(t, s, u)$ jointly over the predictor domain, response domain, and quantile domain using tensor-product B-splines, directional roughness penalties, and an adaptive overlapping group-Lasso penalty.

Usage

```
sfqfofr(
  Xmat,
  Ymat,
  grids,
  bases,
  tuning = FALSE,
  tuning_grid = NULL,
  tuning_criterion = "qbic",
  base_tuning = NULL,
  bic_c = 1.0,
  df_type = "active_coeff",
  integration = c("trapezoid", "simpson", "auto")
)
```

Arguments

- | | |
|--------|---|
| Xmat | Numeric matrix of functional predictors. Each row corresponds to one functional predictor curve and each column corresponds to a grid point in <code>grids\$t</code> . Thus, <code>ncol(Xmat)</code> must equal <code>length(grids\$t)</code> . |
| Ymat | Numeric matrix of functional responses. Each row corresponds to one functional response curve and each column corresponds to a grid point in <code>grids\$s</code> . Thus, <code>ncol(Ymat)</code> must equal <code>length(grids\$s)</code> . |
| grids | <p>A list containing the grids used for fitting. It must contain:</p> <ul style="list-style-type: none"> • <code>t</code>: numeric vector giving the predictor-domain grid; • <code>s</code>: numeric vector giving the response-domain grid; • <code>tau</code>: numeric vector of quantile levels. <p>The quantile grid should contain at least two levels for simultaneous quantile estimation.</p> |
| bases | A list of B-spline basis matrices, typically returned by <code>make_sfqfofr_basis</code> . The list must contain <code>Bt</code> , <code>Bs</code> , <code>Bu</code> , <code>Bsa</code> , and <code>Bua</code> . |
| tuning | Logical value indicating whether tuning-parameter selection should be performed. If <code>FALSE</code> , the estimator is fitted using <code>base_tuning</code> . If <code>TRUE</code> , the candidate vectors in <code>tuning_grid</code> are evaluated and the best one is selected according to <code>tuning_criterion</code> . |

tuning_grid	Optional list of candidate tuning-parameter vectors. Each element should be a named list containing the same components as <code>default_tuning</code> . If NULL and <code>tuning = TRUE</code> , a grid is constructed internally around <code>base_tuning</code> .
tuning_criterion	Character string specifying the tuning criterion. Currently only "qbic" is supported.
base_tuning	Named list of tuning parameters used as the baseline fitting vector. If NULL, <code>default_tuning(nrow(Xmat))</code> is used.
bic_c	Positive numeric multiplier used in the QBIC-type tuning criterion. Larger values impose a stronger penalty on model complexity.
df_type	Character string specifying how the effective degrees of freedom are counted in the QBIC-type criterion. The default is "active_coeff", which counts the intercept coefficients and the selected active slope coefficients after support screening.
integration	Numerical integration rule used to approximate functional inner products. Available options are "trapezoid", "simpson", and "auto". The default is "trapezoid", which is recommended for general grids because it works for arbitrary grid lengths.

Details

The simultaneous function-on-function quantile regression model fitted by this function is

$$Q_{Y(s)|X}(u | X) = \alpha(s, u) + \int_{\mathcal{T}} X(t) \beta(t, s, u) dt, \quad (s, u) \in \mathcal{S} \times \mathcal{U},$$

where $X(t)$ is the functional predictor, $Y(s)$ is the functional response, $\alpha(s, u)$ is the quantile-specific intercept surface, and $\beta(t, s, u)$ is the quantile-varying function-on-function coefficient kernel. The coefficient kernel $\beta(t, s, u)$ describes how the predictor value at location t affects the u th conditional quantile of the response at location s .

The coefficient kernel is approximated by a tensor-product B-spline expansion,

$$\beta(t, s, u) \approx \sum_{a=1}^{K_t} \sum_{b=1}^{K_s} \sum_{c=1}^{K_u} \theta_{abc} b_a(t) c_b(s) d_c(u),$$

where $b_a(t)$, $c_b(s)$, and $d_c(u)$ are B-spline bases over the predictor, response, and quantile domains. The intercept surface is represented separately as

$$\alpha(s, u) \approx \sum_{b=1}^{L_s} \sum_{c=1}^{L_u} \xi_{bc} \tilde{c}_b(s) \tilde{d}_c(u).$$

The finite-dimensional linear predictor is therefore

$$\eta_i(s, u) = \{\tilde{d}(u) \otimes \tilde{c}(s)\}^\top \xi + \{d(u) \otimes c(s) \otimes r_i\}^\top \theta,$$

where

$$r_i = \int_{\mathcal{T}} X_i(t) b(t) dt$$

is the predictor-basis score vector. The integral is computed numerically using the rule specified by `integration`.

Estimation proceeds in three stages. First, a smooth pilot fit is obtained using the empirical check loss together with directional P-spline smoothness penalties and small ridge stabilizers. Second,

an adaptive locally sparse estimator is computed by adding a multiplicity-balanced overlapping group-Lasso penalty to the coefficient tensor. This penalty acts on cuboidal blocks of B-spline coefficients and is designed to identify local regions where the regression kernel is exactly zero. Third, a constrained refit is performed on the selected active coefficient set to reduce the shrinkage bias introduced by the group penalty.

If `tuning = TRUE`, the function evaluates a finite collection of tuning vectors and selects the one minimizing a QBIC-type criterion. The criterion is computed after the full three-stage procedure and penalizes the effective model dimension after support selection. If `tuning = FALSE`, the model is fitted using `base_tuning` directly.

The fitted object stores the pilot, sparse, and refitted estimates. Prediction and coefficient extraction use the final constrained-refit estimator `fit_r`.

Value

An object of class "sfqfofr", represented as a list with the following components:

<code>fit_p</code>	Pilot-stage fit. This is the smooth initial estimator used to construct adaptive group weights.
<code>fit_s</code>	Sparse-stage fit. This is the adaptive locally sparse estimator obtained before the constrained refit.
<code>fit_r</code>	Final constrained-refit estimator. This component is used by <code>predict.sfqfofr</code> and <code>coef.sfqfofr</code> .
<code>tuning</code>	Named list containing the tuning parameters used in the final fit.
<code>tuning_table</code>	If <code>tuning = TRUE</code> , a table containing the evaluated tuning candidates and their criterion values. If <code>tuning = FALSE</code> , this component is <code>NULL</code> .
<code>design_train</code>	Internal sparse design object constructed from the training data.
<code>ginfo</code>	Internal object containing the overlapping group information for the coefficient tensor.
<code>M_grp</code>	Internal group-membership matrix used by the overlapping group penalty.
<code>bases</code>	B-spline basis matrices used in the model fit.
<code>grids</code>	Predictor, response, and quantile grids used in the model fit.
<code>integration</code>	Numerical integration rule used to approximate functional inner products.

Note

The input curves should be observed on the same grids used to construct the basis matrices. If the data are observed on irregular or subject-specific grids, they should first be registered or interpolated to common grids.

The default numerical integration rule is "trapezoid", which is robust for arbitrary grid lengths. Simpson integration can be requested explicitly through `integration = "simpson"`, but it requires an odd number of equally spaced grid points.

The function returns the final constrained-refit estimator as part of a classed object. Use `predict.sfqfofr` for predicted conditional quantile curves and `coef.sfqfofr` for estimated coefficient surfaces.

Author(s)

Ufuk Beyaztas

Examples

```

## Construct grids
t_grid  <- seq(0, 1, length.out = 101)
s_grid  <- seq(0, 1, length.out = 101)
tau_grid <- c(0.10, 0.35, 0.50, 0.65, 0.80, 0.90)

## Build B-spline bases
bases <- make_sfqfofr_basis(
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  kt = 11,
  ks = 11,
  ku = length(tau_grid),
  kas = 5,
  kau = length(tau_grid),
  degree_t = 2,
  degree_s = 2,
  degree_u = 2
)

## Generate a small example data set
dat <- generate_sfqfofr_data(
  n_train = 100,
  n_test = 1000,
  t_grid = t_grid,
  s_grid = s_grid,
  tau_grid = tau_grid,
  bases = bases,
  error_type = "gaussian",
  support_design = "sparse",
  integration = "trapezoid"
)

## Sparse-design-friendly tuning values

tuning_sparse_example <- list(
  lam_init_b = 2.0e-04,
  lam_init_a = 5.0e-05,

  lam_sp_b   = 7.5e-05,
  lam_sp_a   = 3.0e-05,
  lam_sp_g   = 2.5e-02,

  lam_rf_b   = 3.5e-05,
  lam_rf_a   = 2.5e-05,

  active_tol  = 0.32,
  eta_adapt   = 1.5,
  active_method = "fixed_or_gap",
  min_gap     = 0.02
)

## Fit sfqfofr model

```

```

fit <- sfqfofr(
  Xmat = dat$train$X,
  Ymat = dat$train$Y,
  grids = list(t = t_grid, s = s_grid, tau = tau_grid),
  bases = bases,
  tuning = FALSE,
  base_tuning = tuning_sparse_example,
  integration = "trapezoid"
)

## Extract true and estimated beta(t,s,u) at u = 0.50

tau_id <- which.min(abs(tau_grid - 0.50))

beta_true_med <- dat$truth$beta_arr[, , tau_id]
beta_hat_med <- coef(fit, tau_index = tau_id)[, , 1]

## Common z-axis range for fair visual comparison

zlim_common <- range(c(beta_true_med, beta_hat_med), na.rm = TRUE)

zpad <- 0.05 * diff(zlim_common)
if (!is.finite(zpad) || zpad == 0) zpad <- 0.1

zlim_common <- zlim_common + c(-zpad, zpad)

## Plot true and estimated coefficient surfaces side by side

op <- par(
  mfrow = c(1, 2),
  mar = c(2.5, 2.5, 3.0, 1.0),
  oma = c(0.2, 0.2, 0.2, 0.2)
)

persp(
  x = t_grid,
  y = s_grid,
  z = beta_true_med,
  theta = 35,
  phi = 25,
  expand = 0.75,
  col = "gray92",
  border = "gray70",
  ticktype = "detailed",
  xlab = "t",
  ylab = "s",
  zlab = expression(beta(t, s)),
  zlim = zlim_common,
  main = expression(True~~beta(t, s)~~at~~u == 0.50)
)

persp(
  x = t_grid,
  y = s_grid,
  z = beta_hat_med,
  theta = 35,
  phi = 25,

```

```

    expand = 0.75,
    col = "gray92",
    border = "gray70",
    ticktype = "detailed",
    xlab = "t",
    ylab = "s",
    zlab = expression(hat(beta)(t, s)),
    zlim = zlim_common,
    main = expression(Estimated~~beta(t, s)~~at~~u == 0.50)
  )

par(op)

## Predict conditional quantile response curves for test predictors
pred <- predict(fit, Xnew = dat$test$X)

## Dimensions: n_test by length(s_grid) by length(tau_grid)
dim(pred)

## Median quantile index
tau_id <- which.min(abs(tau_grid - 0.50))

## Select the first 10 test curves
id_plot <- 1:10

## True and estimated conditional median curves
Q_true_med <- dat$test$Qtrue[id_plot, , tau_id]
Q_hat_med <- pred[id_plot, , tau_id]

## Plot true and estimated median curves
op <- par(
  mfrow = c(2, 5),
  mar = c(3.0, 3.0, 2.2, 0.8),
  oma = c(0.2, 0.2, 1.0, 0.2)
)

for (ii in seq_along(id_plot)) {

  yy <- range(c(Q_true_med[ii, ], Q_hat_med[ii, ]), na.rm = TRUE)

  plot(
    s_grid,
    Q_true_med[ii, ],
    type = "l",
    lwd = 2,
    ylim = yy,
    xlab = "s",
    ylab = "Median curve",
    main = paste0("Test curve ", id_plot[ii])
  )

  lines(
    s_grid,
    Q_hat_med[ii, ],
    lwd = 2,
    lty = 2
  )
}

```

```
legend(  
  "topright",  
  legend = c("True", "Estimated"),  
  lty = c(1, 2),  
  lwd = c(2, 2),  
  bty = "n",  
  cex = 0.75  
)  
}  
  
mtext(  
  expression("True and estimated conditional median curves, " * u == 0.50),  
  side = 3,  
  outer = TRUE,  
  line = -0.2,  
  font = 2  
)  
  
par(op)
```

Index

`coef (coef.sfqfofr)`, [2](#)
`coef.sfqfofr`, [2](#), [19](#), [24](#)
`default_tuning`, [5](#), [16](#), [17](#), [23](#)
`generate_sfqfofr_data`, [7](#)
`make_sfqfofr_basis`, [8](#), [11](#), [22](#)
`make_tuning_grid`, [6](#), [15](#)
`predict (predict.sfqfofr)`, [18](#)
`predict.sfqfofr`, [3](#), [18](#), [24](#)
`sfqfofr`, [2](#), [5](#), [12](#), [17](#), [18](#), [22](#)